

基于昇腾 910B 的 7B 模型 PPO 后训练项目计划 (v3)

方向收敛为 **模块 A (代码 RLVR) 与 模块 C (工具调用) 二选一**，各自给出从 SFT 开始的全流程任务清单；原指令跟随 / VLM / 数学方向降为附录一句话结论；奖励设计方法论独立成 §3。

0. 决策状态

已敲定：

项	决定
RL 算法	PPO (完整 Actor-Critic)
训练框架	MindSpeed-RL (华为官方, 备选 verl)
基座家族	Qwen2.5, 7B 级, Instruct 版 (变体随方向: 代码方向用 Coder-7B-Instruct, 工具方向用 7B-Instruct)
算力	16 卡 = 2 节点 × 8 卡
奖励范式	RLVR (程序规则判分) 为主链路, 奖励以自设计整形层为核心贡献; 不训 RM (理由见 §3.1)
冷启动	全参轻量 SFT 或 few-shot, 是否需要按 §4/§5 的判定条件实测决定
RLVR 方向	A 代码 与 C 工具调用 二选一 (对比见 §6; 数据可先都下载)

1. 术语速查表 (给全组)

1.1 训练范式与算法

术语	含义
后训练 (post-training)	预训练完成后, 把"会接话的基座"调成"有特定能力的助手"的阶段 (SFT + RL 等)。本项目全部工作在此阶段
基座 / Instruct 版	基座 = 预训练原始模型; Instruct 版 = 已经过指令微调、能正常对话的版本。我们统一用 Instruct 版, 省掉能力训练
SFT (Supervised Fine-Tuning) 监督微调	拿「输入 → 标准答案」数据对模型做监督训练。本项目里 SFT 只干一件事: 统一输出格式 (冷启动)
冷启动	RL 开始前的准备步骤 (轻量 SFT 或 few-shot 提示), 让模型输出格式稳定、奖励判分器能稳定工作
PPO (Proximal Policy Optimization)	本项目用的 RL 算法: 模型生成答案→打分→按优势稳步更新。特点是带一个 Critic 和更新幅度裁剪
GRPO / DAPO	PPO 的变体 (去掉 Critic / 改进采样策略)。昇腾官方示例多用它们, 我们不作为主线, 可选做对照
RLVR (RL with Verifiable Rewards)	可验证奖励: 用 程序规则 (测试通过/答案匹配/约束检查) 产生奖励, 不用神经网络打分。数学、代码、IF 都属此类
RM (Reward Model)	用人类偏好数据训练的打分神经网络。只有"无法程序化判分"的主观任务才需要; 本项目所有方向都可程序化判分, 所以主链路不训 RM (与判分函数的详细对比见 §3.1)

术语	含义
LoRA vs 全参数微调	LoRA 只训练插入的小矩阵（省显存）；全参数 = 所有 7B 权重都更新。我们默认全参，LoRA 做消融对照和显存兜底
消融实验 (ablation)	控制变量的对照实验：只改一个设计点（如奖励里加不加效率分），其余完全相同，跑两次对比，证明该设计有效。 报告原创性的来源
Reward hacking（奖励作弊）	模型钻奖励规则空子拿分但不真正完成任务，例如反复刷格式分、输出超长套话。需要每天人工抽检防它
Curriculum（课程学习）	按难度安排训练数据（如先易后难，或只留"模型做对率 10%~90%"的题——全对/全错的题没有训练价值）
Few-shot prompt	在提示词里放一两个示例，引导模型按固定格式输出
Epoch / Step	全部训练数据过一遍 = 1 epoch；每处理一个 prompt 批次并更新一次 = 1 step
Checkpoint	训练中途保存的模型快照，用于评测、回滚、续训

1.2 PPO 里的五个角色（MindSpeed-RL 会同时部署它们）

角色	是什么	显存里是什么
Actor（策略）	被训练的模型 ，负责生成回复	7B 权重 + 梯度 + 优化器状态
Critic（价值网络）	跟着一起训练的"估分员"，估计当前局面的期望得分，用来算优势、降低更新方差。 它不是 RM	同上，另一份 7B
Reference（参考模型）	冻结的初始模型副本，只读，用来算 KL	只存权重 14GB
Reward（奖励）	我们自己写的判分函数（\$4/\$5），不是模型	不占显存，跑 CPU
Rollout（采样引擎）	高速推理引擎（vLLM 类），批量生成回复	权重 + KV cache

1.3 PPO 常用超参名词

术语	含义
KL 惩罚 (β)	惩罚新模型偏离参考模型的程度，防止训歪、忘掉旧能力。 β 越大越保守，本项目 0.001 起步
GAE（优势估计）	"这次生成的回复比预期好多少"的估计算法； γ 、 λ 是它的两个系数（我们用 1.0/1.0）
clip（裁剪, 0.2）	限制单步策略更新的最大幅度，PPO 稳定性的核心
Rollout n / batch	每个 prompt 生成几条回复（PPO 通常 1 条）；每 step 处理多少 prompt（我们 128~256）

1.4 工程与昇腾生态

术语	含义
MindSpeed-RL / MindSpeed-MM	华为官方 RL 训练框架（本项目用） / 其多模态训练底座
vLLM / vllm-ascend	高吞吐推理引擎及其昇腾适配版；RL 的 rollout 和最终评测都靠它
bf16	半精度数值格式，训练标配，省一半显存
ZeRO / offload	把优化器状态分片到各卡 / 暂时卸载到内存的显存腾挪技术，共卡部署靠它

术语	含义
pass@1 / pass rate	生成一次就判对的比例 / 某模型在某题上的通过率（用于数据难度筛选）

2. 目标与验收

- 主目标：**16 卡上跑通 7B PPO 完整闭环（Rollout → Reward → GAE → Actor/Critic 更新），在所选模块的主评测上取得可量化提升。
- 成功标准：**主评测相对基线 +2pt 以上（基线饱和则以训练动力学 + 消融为主要产出）；训练曲线稳定可解释；**完成 §3.2 预算内的消融：≥2 组完整消融（奖励设计 A/B + PPO vs GRPO）+ KL β 短跑分析。**
- 交付物：**代码与配置、checkpoint、评测表、训练曲线、奖励设计与消融分析、报告 + PPT。

3. 奖励与消融设计（两个模块共用）

$$r = W_{\text{内核}} \cdot \text{core} + W_{\text{整形}} \cdot \text{shaping} - \text{penalties}$$

（内核权重 ≥ 0.7）

core = 可验证内核：测试通过率 / 调用匹配度（客观、程序化）

shaping = 格式分（封顶 0.1~0.2）、部分分、效率/简洁 bonus

penalty = 超长截断归零、解析失败、编译/运行错误

六个设计杠杆（消融选题）：

- 判分粒度：**全有全无 vs 部分分（线性/阶梯）；
- 软硬程度：**硬阈值 vs 连续奖励；
- 组合权重：**多组件 w 的配比；
- 课程设计：**训练池按基座通过率筛选/加权；
- 惩罚结构：**超长、失败的处理方式；
- KL 分工：**什么靠奖励形状引导，什么靠 KL 兜底。

制度：shaping 各项封顶且合计 ≤0.3；奖励函数先过 **100 条人工标注单测，判分一致率 ≥95%** 才准进训练；每 50 step 人工抽检 20 条，监控长度/重复率/格式命中率/正确率四信号防 hacking。

3.1 奖励从哪来：判分函数 vs 训练 RM

PPO 算法本身不关心奖励从哪来——它只负责「拿到分数 → 算优势 → 更新策略」。奖励来源是可插拔的接口：插一个神经网络（RM）就是经典 RLHF（InstructGPT/ChatGPT 的做法），插一个程序就是 RLVR（DeepSeek-R1/o1 的做法）。

	判分函数（我们的主链路）	训练 RM（经典 RLHF 做法）
是什么	确定性程序， 精确 判分	学出来的打分神经网络， 近似 判分
需要的数据	只要题目本身	几千~几万条「同一问题的多条回复 + 人工标注好坏」偏好对
对本任务的适配	代码跑测试 / 调用比对标注 = 直接算出正确性	无现成偏好数据；造标签得先用规则判分器，绕一圈回到原点
成本	零训练，写好即插即用	7B 版 1~2 天 8 卡；还引入新的 reward hacking 面

	判分函数（我们的主链路）	训练 RM（经典 RLHF 做法）
什么时候必须用	——	仅当任务无法程序化判分（如“哪个回答更有帮助/更安全”）才必须

一句话结论：对可验证任务，RM 是「把能精确计算的量换成有噪近似」，而且设计工作并没有消失、只是转移到更麻烦的偏好数据构造上。注意我们并没有去掉“模型”——Critic 仍是在训的 7B 模型——只是不让模型来打分。「自己设计奖励」指的就是设计这个判分函数，设计空间见 §4.3 / §5.3 的 ★ 点。

3.2 消融菜单与算力预算（全项目）

原则：**做过选择的地方就有消融**（单变量对照：把该选择换成对照项、其余不动、跑一遍对比）。每组“完整消融”≈ 1 次过夜训练，16 卡一周的预算 ≈ **1 次主训练 + 2 次完整消融 + 若干短跑**。

层	选题	成本	价值	决定
奖励设计	A：部分分策略（线性 / 阶梯 / 全有全无）；C：权重配比 2~3 组	1 次完整训练（节点 2）	★★★ 原创性的直接证据	必保
算法	PPO vs GRPO ：同数据同奖励，MindSpeed-RL 改配置即可切换；GRPO 无 Critic，8 卡可跑	1 次完整训练（8 卡，与主训练并行）	★★★ 呼应“为什么用 PPO”	必保
冷启动	LoRA vs 全参 SFT	下游需再跑一次 PPO	★★ 工程意义大于科研意义	可砍
数据	难度过滤 on/off（全量池 vs pass rate 10%~90% 池）	1 次完整训练	★★ 课程学习证据	备选
超参	KL β 0.0005 vs 0.005	短跑 20~30 step	★ 训练动力学分析	D3 顺手做

两条纪律：**短跑消融只能写“训练动力学”分析，不得下“最终性能更好”的结论**（评审会抓）；报告里 2~3 组“设计 → 结果 → 解释”完整的消融，胜过 5 组只有数字的。

4. 模块 A：代码 RLVR —— 全流程任务清单

4.1 任务定义与训练样本示例

给模型一道编程题，模型生成 Python 代码，**代码进沙箱跑单元测试，通过率即奖励**。练的是“写出一次通过测试的代码”。

【一条训练数据长什么样】

Prompt（进模型）：

请实现函数 `solve(nums, target)`：返回 `nums` 中和为 `target` 的两个数的下标。
先简短思考，再把完整代码放在 ``python 代码块中。

模型生成（Rollout 采样）：

（思考：用哈希表一次遍历.....）

```
``python
def solve(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen: return [seen[target-n], i]
        seen[n] = i
    ``
```

奖励计算（我们写的，跑在 CPU 沙箱）：

提取代码块 → 跑 5 条断言（限时 10s）→ 4/5 通过
 $r = 0.1(\text{格式}) + 0.9 \times 4/5 = 0.82$

基座：Qwen2.5-Coder-7B-Instruct（先下载；通用版 7B-Instruct 可作对照）。

4.2 六阶段流水线（从 SFT 开始）

阶段	做什么	产出 / 验收
① 环境 (D1)	官方 Docker 镜像装 MindSpeed-RL；小模型 PPO 冒烟；实测 7B 共卡显存/单步耗时	Go/No-Go：小模型 PPO 跑通
② 冷启动 SFT (D2, 按判定条件取舍)	判定： 先拿 50~100 道池内题测基座裸格式命中率（输出含 ``python 块）， $\geq 90\%$ → 跳过 SFT 用 few-shot 模板； $< 90\%$ → 做 SFT：从 KodCode-Lite 抽 1~3k 条（题目 + 官方验证解答），程序化重排成与 RL 完全相同的模板，全参训练 1e-5、2 epoch、seq len 2048，8 卡 1~2h	SFT checkpoint；格式命中率 $\geq 95\%$ ；HumanEval 抽样不降（防 SFT 打伤能力）
③ 数据池构建 (D2)	KodCode-Lite 抽 5~8k → 提取并验证单测（官方测试也要试跑，单题 $< 10s$ ）→ 用基座全池预采样算通过率 → **留 pass rate 10%~90% 的题 2~4k 条**为 RL 池（全对/全错剔除）	r1_pool.jsonl (prompt + tests 元数据)；TACO 中等难度子集补充
④ 奖励模块 (D2, 与③并行)	写三件套：extract_python（代码块提取）+ run_in_sandbox（subprocess + timeout 10s + 内存 rlimit + 断网 + 并发池，放 CPU 节点）+ 奖励组装函数（按 §3 三层结构）；100 条边界单测（无代码块/死循环/超时/多代码块）人工对齐 $\geq 95\%$	reward/ 模块；对齐报告
⑤ PPO 训练 (D3 调通, D4~D5 主训)	D3：128 prompts × 20 step 小规模验证显存/吞吐/reward 上行；D4~D5：全量 2~4k prompts × 1~2 epoch（约 30~60 step）过夜；max_new_tokens 768（思考+代码比数学长）；每 10~20 step 存 checkpoint	训练曲线：reward、correct 率、长度、格式率、KL、熵
⑥ 评测与消融 (D6)	主评测 LiveCodeBench （时间切分防泄漏，抽 200~300 题 greedy pass@1）；辅评 MBPP+ 防退化；消融按 §3.2 预算：奖励 A/B = 部分分策略（线性 / 阶梯 / 全有全无，主推）节点 2 完整跑；PPO vs GRPO 8 卡并行；（可选）LoRA vs 全参	评测表 + 消融曲线 + 抽检 50 条

4.3 自设计奖励落点（★ 选 1~2 个做成消融）

- 部分分策略**（主推）：通过 3/10 个测试给 0.3 还是 0.5 还是 0？线性 / 阶梯 / 全有全无三组对照；
- 效率奖励**：全过的代码按运行时间给 bonus（快的 +0.1 封顶），引导高效代码——文献少见，好讲故事；

- 3. 失败惩罚结构：编译失败 0 分 vs 负分 vs 只扣格式分；
- 4. 课程：训练中段把低通过率题的采样权重调高。

A 模块要点提醒：奖励是真实执行出来的，客观性是所有方向里最强的——报告里要主动强调这一点。

5. 模块 C：工具调用 —— 全流程任务清单

5.1 任务定义与训练样本示例

给模型一段用户请求和一系列可用函数（JSON schema），模型输出一次函数调用。练的是"选对函数、填对参数、格式合法"。输出很短（几十个 token），是两个模块里 PPO 循环最快的。

【一条训练数据长什么样】

Prompt（进模型）：

你可以调用以下函数：

[{"name": "get_weather", "description": "...", "parameters": {"city": "string", "date": "ISO8601"}}, {"name": "search_train", ...}, {"name": "book_hotel", ...}] （共 3 个）

用户：北京明天天气怎么样？帮我安排下。

请输出一个 <tool_call>{...}</tool_call>。

模型生成（Rollout 采样）：

<tool_call>{"name": "get_weather", "arguments": {"city": "北京", "date": "2026-09-13"}}</tool_call>

奖励计算（四级分解加权）：

r = 0.15×格式(JSON 合法且符合 schema)

+ 0.30×选函数(函数名与标注一致)

+ 0.55×参数(应写 2 个字段、写对 2 个 → 1.0；缺/错一个按比例扣)

本例全对 → r = 1.0；若 date 缺失 → 0.15+0.30+0.275 ≈ 0.73

基座：Qwen2.5-7B-Instruct。注意一个客观局限：xLAM 数据没有真实执行环境，“参数对不对”是与标注答案匹配（近似正确性，不是真执行）——报告里如实说明，或者用 BFCL 的可执行子集补强。

5.2 六阶段流水线（从 SFT 开始）

阶段	做什么	产出 / 验收
① 环境 (D1)	同模块 A（两模块共用环境，先都装好）	Go/No-Go
② 冷启动 SFT (D2, 此模块大概率需要)	判定：拿 100 道池内题测基座"解析成功率"（输出能被 <tool_call> 正则提取且 JSON 合法），<80% → 必须做 SFT。做法：xLAM-60k 抽 2~3k（query + 可用函数列表 + 标注调用），程序化重排成与 RL 相同模板，全参 lr 1e-5、2 epoch，8 卡 1~2h。SFT 的意义比模块 A 大：函数调用格式严格，通用基座服从度不稳	SFT checkpoint；解析成功率 ≥95%；BFCL AST 抽样不降
③ 数据池构建 (D2)	xLAM-60k 清洗：去重、过滤超长函数列表（>8 个函数的截掉或弃用）、校验标注调用与 schema 一致（日期/枚举值规范化）；按"函数个数、参数嵌套深度"分层抽 3~5k 为 RL 池，留 500 条做 held-out	rl_pool.jsonl + heldout.jsonl
④ 奖励模块 (D2, 与③并行)	解析器（<tool_call> 提取 + json.loads + schema 校验）+ 四级评分函数；100 条边界单测（非法 JSON/幻觉函数名/缺参数/多余参数/多调用）对齐 ≥95%	reward/ 模块

阶段	做什么	产出 / 验收
⑤ PPO 训练 (D3 调通, D4~D5 主训)	同 A 的节奏; max_new_tokens 256 (输出短, 单步最快, 同样算力下能跑更多 step)	训练曲线同 A
⑥ 评测与消融 (D6)	主评测 BFCL (Berkeley Function-Calling Leaderboard 的 AST/简单匹配类) + 自建 held-out 500 题; 消融按 §3.2 预算: 奖励 A/B = 权重体系 (0.15/0.30/0.55 vs 提高格式分权重, C 模块最出彩) 节点 2 完整跑; PPO vs GRPO 8 卡并行	评测表 + 消融曲线

5.3 自设计奖励落点 (★ 选 1~2 个做成消融)

- 1. **权重体系** (主推): w格式/w函数/w参数 怎么配? 做 2~3 组配比对照, 画出"学习速度 vs 最终效果"的权衡——天然消融矩阵;
- 2. **参数部分分粒度**: 逐字段给分 vs 整体对错;
- 3. **错误结构**: 幻觉函数名 (调了列表里不存在的函数) 和参数错, 哪个该罚更重?
- 4. **多调用题**: 需要连续调用两个函数的样本怎么给分 (可作扩展池)。

6. 模块 A vs 模块 C: 怎么选

维度	A 代码	C 工具调用
奖励客观性	最强 (真实执行单测)	中 (与标注匹配, 无真执行)
数据工程	较重: 沙箱 1 天 + 测试验证	中: 清洗 + 格式化约 1 天
奖励工程	中: 执行器 + 组装	中: 解析器 + schema 校验
训练吞吐	中 (生成 768 token)	快 (生成 256 token, 同卡时可多跑 step)
评测	最标准 (LiveCodeBench 现成)	标准 (BFCL 现成) + 需自建 held-out
SFT 必要性	低 (Coder 版格式好, 实测≥90% 可跳)	较高 (调用格式严格, 实测大概率先 <80%)
报告故事	"RLVR + 真实执行的奖励 + 课程设计", 经典扎实	"四级奖励分解 + 权重消融", 设计感强、差异化好
主要风险	沙箱工程卡壳 (预案: 先只跑 MBPP 级简单测试)	解析失败率过高吃掉有效梯度 (预案: 格式分 + few-shot 拉格式)

选择建议: 想要最稳、奖励最硬核、评测最省心 → **A**; 想要奖励结构设计的差异化故事、避开沙箱、训练循环更快 → **C**。两模块的 ①环境/③流程骨架/⑤PPO 配置完全共享, **D2 早晨用"谁的判分器先通过 100 条单测"作为最终拍板依据**, 数据两个都先下载 (KodCode-Lite 约 20GB、xLAM-60k 数 GB)。

7. 框架、部署与资源 (16 卡)

- MindSpeed-RL: 官方 Docker 镜像; 算法覆盖 PPO 全角色; 文档要求 Actor/Reference 用 SFT 后模型启动, 与冷启动设计一致。仓库 github.com/Ascend/MindSpeed-RL (Gitee 镜像同名), 配套昇腾社区《LLM 后训练》教程。备选 verl (D1 双路线并行, 先通先用)。

- 节点 1 (8 卡)：PPO 主线，训推共卡 (Actor+Critic+Reference+Rollout 分时复用，靠 offload/ZeRO 腾显存)。
- 节点 2 (8 卡)：D2 跑 SFT (1~2h) 与基线评测；此后常驻评测推理 + 串行消融。
- CPU 资源：A 模块沙箱批量执行、C 模块解析都在 CPU 节点/管理节点跑，不占卡。
- OOM 降级链：加大 offload → ZeRO-3 → 16 卡训推分离 (SFT/评测与训练错峰) → LoRA。

8. 训练超参起点

项	起点值
prompts/step	128~256 (PPO 每 prompt 采 1 条)
采样温度	1.0
max_new_tokens	A: 768; C: 256
PPO clip / GAE	0.2; $\gamma=1.0$, $\lambda=1.0$
Actor lr	1e-6, cosine + warmup 3~5 步 (崩溃降 5e-7)
Critic lr	5e-6 ~ 1e-5
KL β	0.001 (累计 KL 目标 <10 nats)
训练量	A: 2~4k prompts $\times$ 1~2 epoch; C: 3~5k $\times$ 1~2 epoch
SFT (若做)	lr 1e-5、2 epoch、bf16、全参、8 卡 1~2h
吞吐预估	8 卡共卡约 2~5 min/step (D1 实测回填)

监控项：reward 均值/方差、判对率、格式命中率、KL、策略熵、平均长度、critic loss。

9. 一周日程

天	目标	内容
D1	框架通	官方镜像 → 小模型 (0.5B) PPO 冒烟 → 7B 显存/耗时实测 → Go/No-Go (不通切 verl)。下载 KodCode-Lite + xLAM-60k
D2	数据与奖励	两模块并走半天：数据清洗 + 判分器 100 条单测 (谁先过 95% 谁上，早晨拍板) → 基线评测 → 冷启动 SFT (按判定条件)
D3	PPO 调通	7B 小规模 20 step：显存、吞吐、reward 上行、KL/长度正常；顺手跑 KL β 短跑消融 (§3.2)
D4~D5	主训练	全量 1~2 epoch 过夜；每日评 checkpoint；节点 2 跑两组完整消融：奖励 A/B + PPO vs GRPO (8 卡，见 §3.2)
D6	消融与评测	(可选) LoRA vs 全参消融，时间紧则砍；全量评测 + 人工抽检 50 条
D7	收尾	报告 (重点：奖励设计 + 消融结论)、PPT、代码整理；留半天缓冲

分工明细 (4 人，两条线：工程线 = P1/P2，研究线 = P3/P4；名字自行对号)

人	角色	主责	关键交付
P1	工程负责人	环境安装与框架选型（D1 双路线并行）、7B 调通、主训练值机（显存/吞吐/断点/OOM 处置）	D1 Go/No-Go 结论；调通的训练配置；主训练 checkpoint
P2	数据负责人	数据四道筛（格式适配 / 判分验证 / 难度过滤 / 去重）、判分内核代码（A=沙箱 或 C=解析器）	<code>rl_pool.jsonl</code> + <code>heldout.jsonl</code> ； <code>reward/</code> 模块 + 100 条单测对齐报告
P3	研究/实验负责人	奖励三层结构与 ★ 设计点方案、消融矩阵设计与执行、基线与全量评测	奖励设计文档（含每个 ★ 点的消融假设）；评测表与消融曲线分析
P4	报告与质量	防 hacking 抽检轮值、训练曲线整理、报告与 PPT 主笔、机动支援	每日抽检记录；报告初稿（D6 起）

一周里谁最重：

- **D1：全员装机**（环境是关键路径，四个人都要能跑命令），P1 主导；P2 同时挂数据下载；
- **D2**：P2+P3 并行攻判分器（P2 写内核、P3 定设计与单测）→ 早晨据此拍板 A/C；P1 基线评测；P4 建抽检制度；
- **D3**：P1 调通 7B 主线；P3 顺手做 KL β 短跑；P2 清数据尾巴；
- **D4~D5**：P1 主训练值机；P3 在节点 2 跑两组完整消融；P2 随时修数据侧问题；P4 整理曲线 + 写报告初稿；
- **D6**：P3 全量评测 + 消融收尾；P4（可选）LoRA 消融 + 报告成稿；P1 机动支援；
- **D7**：全员报告与 PPT 收尾。

共同约定：每日 15 分钟站会（训练状态 / 消融进度 / 新风险三件事）；所有实验配置与脚本进 git；抽检轮值表由 P4 排。

里程碑砍法（从上往下保）：最小闭环（7B+PPO+自设计奖励+主评测+曲线）→ 奖励 A/B 消融 → PPO vs GRPO 消融 → LoRA 消融 → 第二个奖励设计点。

10. 风险登记册

#	风险	信号	预案
1	版本兼容（CANN/torch_npu/MindSpeed 配对）——最高风险	装不上、算子报错	只用官方镜像；D1 双框架并行，先通先用
2	(A) 沙箱卡死/资源耗尽	执行队列堆积	timeout+rlimit+并发上限；死循环题剔除
3	(A) 数据泄漏使评测虚高	LiveCodeBench 之外基线异常高	主评测只用时间切分集
4	(C) 解析失败率过高	格式分占比异常、有效梯度稀	SFT 拉格式 + few-shot；解析失败=0 分不重试
5	(C) 标注噪声（xLAM 参数不规范）	单测对齐率低	清洗规范化（日期/枚举）；对齐 <95% 不进训练
6	PPO 崩溃（reward 崩/KL 爆/长度爆）	曲线突变	降 lr 5e-7、提 β 、回滚 checkpoint、过滤超长
7	共卡 OOM	显存超 64GB	\$7 降级链

#	风险	信号	预案
8	吞吐不达标 (>5 min/step)	1 epoch 超 8h	砍 max_new_tokens、减 prompts/step、收紧数据池
9	基线饱和提升不显著	基线 >70~80 分	换更难数据池/评测；报告以奖励消融为主要贡献
10	reward hacking	长度/格式分异常涨、正确率不涨	§3 防作弊清单制度化执行

11. 快速索引

方向 A: [KodCode/KodCode-Lite](#)（主池） | [google-research-datasets/mbpp](#)（冒烟） | [BAAI/TACO](#)（补难度） | 评测 [evalplus/humanevalplus](#)、[evalplus/mbppplus](#)、[LiveCodeBench](#)（时间切分） **方向 C:** [Salesforce/xlam-function-calling-60k](#)（主池） | 评测 [BFCL](#) ([gorilla.cs.berkeley.edu](#)) | 方法论 [ToolRL](#) (arXiv:2504.13958)

通用: [github.com/Ascend/MindSpeed-RL](#) | 昇腾社区《LLM 后训练》教程 | [github.com/volcengine/ver1](#) | [DeepSeek-R1](#) (arXiv:2501.12948) | [TÜLU 3](#) (arXiv:2411.15124)